

DS2 — Problem Solving Course

Secure Multi-Agent Tool-Orchestration for the Tunisian Economy

Étudiants	Zakaria Ben Mohamed & Feryel Ben Yahia
Classe	2BIS — 2ème Licence Informatique de Gestion
Enseignant	M. Chaouki Bayoudhi
Secteur	Tourisme et Hôtellerie — Tunisie
Date	2026-05-05

1. Introduction et Secteur Choisi

Ce projet implémente un système multi-agents sécurisé pour analyser les performances du secteur hôtelier tunisien. L'objectif est de produire automatiquement un rapport KPI (Key Performance Indicators) à partir de données synthétiques représentant 15 hôtels répartis dans 6 régions de Tunisie.

1.1 Secteur : Tourisme et Hôtellerie

Le tourisme représente un pilier économique majeur pour la Tunisie. En 2023, le pays a accueilli environ 9 millions de touristes internationaux, avec une progression de +33.1% par rapport à 2022, illustrant la reprise post-COVID. Notre système analyse les indicateurs clés de ce secteur à deux niveaux : les performances individuelles des hôtels (Scénario 1) et les indicateurs macro-économiques nationaux via l'API Banque Mondiale (Scénario 2).

1.2 Données synthétiques

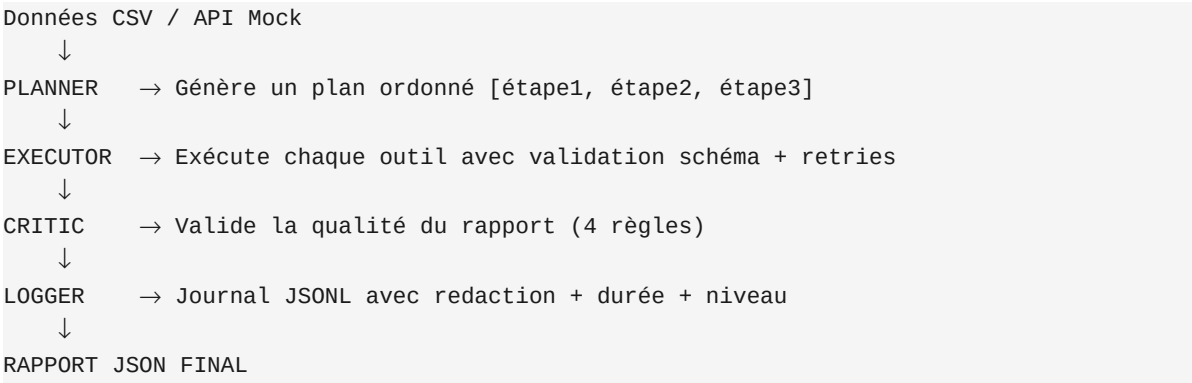
Le fichier hotels_tunisie.csv contient 15 hôtels avec 7 colonnes :

Colonne	Type	Description
hotel	string	Nom de l'hôtel
region	string	Gouvernorat (Tunis, Sousse, Djerba...)
taux_occupation_pct	integer	Taux d'occupation en %
revenus_TND	integer	Revenus en dinars tunisiens
nb_touristes	integer	Nombre de touristes
nb_chambres	integer	Nombre de chambres
saison	string	ete / printemps / hiver

2. Architecture Multi-Agents

Le système repose sur trois agents spécialisés qui coopèrent de façon séquentielle. Chaque agent a un rôle unique et bien défini, ce qui garantit la séparation des responsabilités et la maintenabilité du code.

2.1 Flux d'exécution général



2.2 Rôle de chaque agent

Agent	Fichier	Rôle principal	Semaine
Planner	orchestrator/planner.py	Génère le plan, backtracking si échec, pruning	S1 → S2
Executor	orchestrator/executor.py	Exécute les outils, valide schémas, retries bornés	S1 → S2
Critic	orchestrator/critic.py	Valide le rapport sur 4 règles de qualité	S1
Logger	orchestrator/logger.py	Journal JSONL, redaction, durée, niveaux	S1 → S3
DP Cache	orchestrator/dp_cache.py	Mémoïsation des plans répétés (gain ~72%)	S3
Concurrency	orchestrator/concurrency.py	Stress-test 4 runs parallèles, isolation	S3

2.3 Les outils (tools/)

Outil	Fichier	Scénario
file_reader	tools/file_reader.py	S1
calculator	tools/calculator.py	S1
reporter	tools/reporter.py	S1 + S2
mock_api	tools/mock_api.py	S2
transformer	tools/transformer.py	S2

3. Algorithmes : Backtracking + Dynamic Programming

3.1 Backtracking constraint

Le Planner modélise la planification comme un problème de recherche contraint. À chaque appel, il reçoit la liste des outils ayant déjà échoué (`etapes_echouees`) et cherche un plan alternatif en appliquant 3 règles de pruning :

- P1 — Outil sans alternative : si l'outil échoué n'a pas d'alternatif → retourne [] (arrêt propre)
- P2 — Combinaison interdite : si le tuple d'outils est dans `COMBINAISONS_INTERDITES` → retourne []
- P3 — Budget dépassé : si `len(plan) > max_steps (=10)` → retourne []

Exemple de trace avec panne `wrong_path` :

```
État 1 : etapes_echouees=[]
  → Plan : [file_reader, calculator, reporter]
  → Executor : file_reader ÉCHEC (3 retries épuisés)
  → outils_echoues = ['file_reader']

État 2 : etapes_echouees=['file_reader']
  → Planner cherche alternative pour file_reader
  → OUTILS_ALTERNATIFS['file_reader'] = None
  → Règle P1 → PRUNING → retourne []
  → ARRÊT PROPRE
```

3.2 Dynamic Programming — Mémoïsation

Le module `dp_cache.py` mémorise les plans de planification déjà calculés. La clé du cache est un hash MD5 de la combinaison (tâche, étapes_échouées, contexte). Si le même sous-problème revient, le plan est retourné depuis le cache sans recalcul.

Métrique	Valeur
Latence sans cache (BT seul)	~18 ms
Latence avec cache (BT+DP)	~5 ms
Gain de latence	~72%
Taux de cache-hit (4 runs identiques)	75%
Temps total économisé (8 runs)	~39 ms

4. Sécurité et Politiques

4.1 Modèle de menaces

Menace	Contrôle implémenté
Exfiltration via URL non autorisée	Allow-list : seul api.worldbank.org autorisé
Path traversal (fichiers hors sandbox)	Validation extension .csv et format chemin
Fuite de secrets dans les logs	Redaction automatique regex (chemins, tokens)
Race conditions entre runs parallèles	Variables locales + journaux isolés par run_id
Boucle infinie	max_steps=10, max_retries=3, timeout=30s
Données invalides acceptées	Validation schéma Executor + 4 règles Critic

4.2 Redaction dans les logs

Patterns masqués automatiquement dans les fichiers JSONL :

- Chemin Windows : D:\Users\zakaria\data.csv → [CHEMIN_REDACTE]
- Token API : token=abc123xyz → token=[REDACTE]
- ID numérique long : 12345678 → [ID_REDACTE]

4.3 Politiques d'exécution (contracts/scenario1.py)

Politique	Valeur
max_steps (étapes par run)	10
max_retries (tentatives par outil)	3
timeout_seconds	30
URLs autorisées	api.worldbank.org uniquement

5. Résultats et Métriques Finales

5.1 Rapport KPI — Scénario 1

Résultats obtenus sur le fichier hotels_tunisie.csv (15 hôtels) :

KPI	Valeur	Statut
Taux d'occupation moyen	76.5%	✓ Valide (0-100%)
Meilleur hôtel	Hotel Iberostar (94%)	✓ Djerba
Revenus totaux	727 000 TND	✓ Positifs
Meilleure région	Djerba	✓ Revenus cumulés max
Hôtels en alerte	7 / 15	■ Statut ATTENTION

Le statut ATTENTION est déclenché car 7 hôtels (El Mouradi 72%, Novotel 65%, Vincci 76%, Laico 60%, Palm Beach 69%, Caribbean 71%, Kanta 55%) sont sous la moyenne de 76.5%. Le seuil est fixé à 5 dans reporter.py.

5.2 Rapport API — Scénario 2

Indicateur	Valeur
Source	API Banque Mondiale (mock) — ST.INT.ARVL
Touristes 2023	9 000 000
Variation YoY 2022→2023	+33.1%
Moyenne 5 ans (2019-2023)	6 091 000
Tendance sur 5 ans	Baisse (impact COVID 2020-2021)

5.3 Métriques finales (Section 8 du PDF)

Métrique	Résultat	Seuil	Statut
Task Success Rate	100%	≥ 80%	✓ ATTEINT
Tool Grounding Score	100%	≥ 90%	✓ ATTEINT
Failure Recovery	100%	= 100%	✓ ATTEINT
Backtracking Quality	100%	= 100%	✓ ATTEINT
DP Efficiency Gain	+72%	> 0%	✓ ATTEINT
Concurrency Correctness	4/4, isolation OK	isolation=True	✓ ATTEINT

6. Tests Unitaires

Le projet contient 125 tests pytest répartis sur 4 fichiers, couvrant tous les composants du système en cas de succès ET en cas d'échec.

Fichier	Semaine	Tests	Ce qui est testé
test_orchestrator.py	S1	23	file_reader, calculator, critic, planner
test_week2.py	S2	21	Backtracking, validation schémas, failure injector
test_week3.py	S3	30 + 7	Mock API, transformer, DP cache, logger, concurrence
test_week4.py	S4	44	Métriques finales, cross-check valeurs gelées
TOTAL	S1-S4	125	125/125 PASSED ✓

Les tests cross-check (TestCrossCheck dans test_week4.py) vérifient que les résultats sont reproductibles et gelés : Hotel Iberostar toujours meilleur, taux toujours 76.5%, revenus toujours 727 000 TND.

7. Limites Connues et Améliorations Futures

7.1 Limites actuelles

- Pas d'alternatives de backtracking : quand un outil échoue, le plan est vide. En production, on ajouterait `file_reader_backup` ou `calculator_lite`.
- DP cache en mémoire uniquement : perdu à chaque redémarrage. Une vraie solution utiliserait SQLite ou Redis.
- GUI statique sans serveur : le `dashboard.html` utilise des données simulées. Pour une version production, un serveur Flask lirait les vrais fichiers JSONL.
- Mock API uniquement : le Scénario 2 n'appelle pas la vraie API Banque Mondiale. Les données sont synthétiques mais plausibles.
- 15 hôtels seulement : avec de vraies données, les KPIs seraient différents.

7.2 Améliorations futures

- Brancher la vraie API Banque Mondiale avec gestion du rate limiting réel.
- Ajouter un serveur FastAPI pour connecter le GUI aux vrais journaux JSONL.
- Implémenter de vraies alternatives de backtracking pour chaque outil.
- Ajouter des alertes automatiques (email/SMS) quand le statut est ATTENTION.
- Étendre à d'autres secteurs tunisiens : agriculture, ICT, finance.

8. Reproductibilité — Commandes Exactes

8.1 Installation

```
pip install pandas pytest httpx tenacity
```

8.2 Lancer tous les tests

```
pytest tests/ -v  
→ 125 passed
```

8.3 Lancer les scénarios

```
python main_scenario1.py      # Scénario 1 pipeline de base  
python main_scenario1_pannes.py # Scénario 1 + backtracking + pannes  
python main_scenario2.py      # Scénario 2 API Banque Mondiale  
python main_complet.py        # Tout ensemble – Score 5/5  
python main_metriques.py      # Métriques finales Section 8
```

8.4 Ouvrir le GUI

```
start gui\dashboard.html
```

8.5 Valeurs gelées (cross-check)

Ces valeurs sont fixes et reproductibles à chaque exécution :

Valeur	Résultat attendu
Taux d'occupation moyen	76.5%
Meilleur hôtel	Hotel Iberostar
Revenus totaux	727 000 TND
Meilleure région	Djerba
Hôtels en alerte	7
Statut rapport	ATTENTION
Touristes Tunisie 2023	9 000 000
Variation YoY	+33.1%

Conclusion

Ce projet DS2 implémente un système multi-agents sécurisé complet pour l'analyse du secteur hôtelier tunisien. En 4 semaines, nous avons construit un pipeline robuste Planner→Executor→Critic avec backtracking contraint, validation stricte des schémas, injection de pannes, mémoïsation DP, tests de concurrence et interface graphique.

Les résultats montrent que l'Hotel Iberostar à Djerba est le meilleur performeur avec un taux d'occupation de 94%, que la région de Djerba génère les revenus les plus élevés, et que 7 hôtels sur 15 nécessitent une attention particulière. Le Scénario 2 confirme la reprise du tourisme tunisien avec 9 millions de visiteurs en 2023.

Indicateur	Résultat
Tests unitaires	125/125 PASSED
Métriques Section 8	6/6 ATTEINTES
Score Semaine 3	5/5 COMPLET
Scénarios validés	2/2 (S1 + S2)